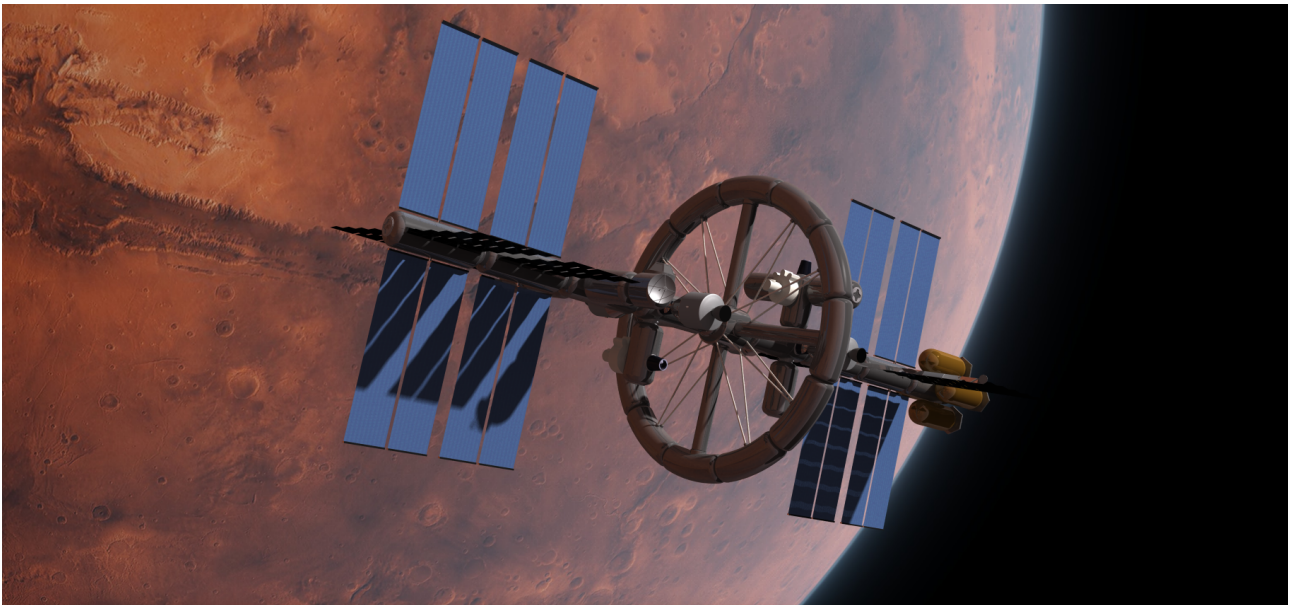


Design of an Extended Kalman Filter and a Nonlinear Attitude Controller for the Arceus Mars Vehicle

MTV04 Individual Report



Academic supervisor: Dr. Aaron Knoll
Prof. Silvestre Pinho
Department: Department of Aeronautics
Course: H415 MEng in Aeronautics
Module: AERO96005 - Group Design Project
Project: MTV - Mars Transit Vehicle
Academic year: 2020/2021

Student: Maria Luisa Scarpa
CID: 01496250
Date: 14/06/2021

Department of Aeronautics
South Kensington Campus
Imperial College London
London SW7 2AZ
U.K.

Abstract

The Attitude Determination and Control System of the *Arceus*, a large crewed Mars vehicle, is designed. This report will outline the procedure followed in the realisation of an Extended Kalman Filter for attitude determination. In order to accurately correct the position of the vehicle in space and counteract external disturbances, a linear controller is not sufficient, thus, non linear control techniques will be explored. Finally, the possibility of using Control Moment Gyroscopes, so as to meet actuators torque requirements, will be investigated. Because of the nature of this project, teamwork and intercommunication between sub-teams was crucial for the success of the mission. Even though the work outlined in this report has been conducted individually, the reader will be referred to the work of relevant team members when needed.

Keywords— Extended Kalman Filter, Lyapunov Control Law, Non Linear Controller, Control Moment Gyroscopes, Teamwork

Contents

List of Figures	iv
List of Tables	iv
List of Symbols	v
1 Introduction	1
1.1 Motivation	1
1.2 Overview & objectives	1
2 System dynamical equations	1
3 Extended Kalman Filter	1
3.1 Principle	1
3.2 Mathematical Process	2
3.2.1 Propagation Cycle	2
3.2.2 Correction Cycle	2
3.3 Simulation & Testing	3
3.3.1 Parameters initialisation and choices	3
3.3.2 Results	4
4 Attitude Control	6
4.1 Introduction to Nonlinear control techniques	6
4.1.1 Lyapunov control law	6
4.1.2 Feedback linearisation	6
4.2 Nonlinear PID controller design	6
4.2.1 Controller selection	6
4.2.2 Controller design	6
4.2.3 Unmodeled disturbances & Saturation	7
4.3 Simulation & Testing	7
5 Control Moment Gyroscopes (CMGs)	8
5.1 Actuators configuration	8
5.1.1 Overview	8
5.1.2 CMGs configuration	9
5.2 CMGs mathematical model	9
5.2.1 Steering law	9
5.2.2 Singularity avoidance	10
5.3 Simulation & Testing	10
5.4 Failure analysis	11
6 Discussion	11
7 Outlook	12
References	13
Appendices	15
A Supplementary Plots	15
B MATLAB Codes	16
B.1 EKF function	16
B.2 ODE solver	17

B.3	Main Kalman filter script	19
B.4	Non linear controller	21
B.5	CMGs	25

List of Figures

1	Attitude determination and control block diagram	2
2	Comparison between real and estimate change of ω with time	4
3	Comparison between real and estimate change of q with time	5
4	Angular velocity change with time	8
5	Attitude quaternion error change with time	8
6	Control input torque with time when an unconstrained control law is implemented . .	8
7	Control input torque with time when a constrained control law is implemented	8
8	CMGs placed in a pyramid configuration [1]	9
9	Gimbal angles required for the maneuver in an unconstrained case	10
10	Gimbal angles required for the maneuver in a constrained case	11
11	Comparison between attitude error over time and square root of diagonal entries of P matrix	15

List of Tables

1	Comparison between mean value of P matrix entries and error in q	5
2	Comparison between mean value of P matrix entries and error in ω	5
3	Parameters chosen for controller simulation	7
4	Comparison between singularity avoidance methods as presented in [2]	10

List of Symbols

$\delta\omega$	Angular velocity error $\omega - \omega_r$ [rad/secs]
Δt	time step $t_{k+1} - t_k$ for Extended Kalman Filter [secs]
$\omega = [\omega_x, \omega_y, \omega_z]^T$	Body rate with respect to the inertial frame [rad/secs]
Φ	State transition matrix [-]
$e = x - x_r$	Error state vector [-]
g	Gimbal angles of the CMGs [deg]
h_{cmg}	Angular momentum stored in the Control Moment Gyroscopes [Nms]
J	Spacecraft's moment of inertia matrix [kg · m ²]
K	Kalman filter gain [-]
P	Error covariance matrix [-]
$q = [q_1, q_2, q_3, q_4]^T$	Attitude quaternion with respect to the inertial frame [-]
Q	Process noise covariance matrix [-]
R	Observation noise covariance matrix [-]
R_{bi}	Rotational matrix from inertial to body frame [-]
s_b	Sun vector in body frame [-]
s_i	Sun vector in inertial frame [-]
T_a	Spacecraft's actuators torques [N · m]
T_d	Spacecraft's disturbances torques [N · m]

$u(t)$	General control input [-]
$x_{(k+1 k)}$	Estimate of the state at time t_{k+1} before measurements processing [-]
$x_{(k+1 k+1)}$	Estimate of the state at time t_{k+1} after measurements processing [-]
$x_{(k k)}$	Estimate of the state at time t_k [-]
z_B	Measurements obtained directly from onboard sensors in body frame [-]
z_i	Inertial frame references from model prediction [-]
β	Skew angle of the CMGs configuration [deg]
ΔT	Unmodeled external torques [Nm]
ϵ	Quaternion error $[q_{1e}, q_{2e}, q_{3e}]^T$ [-]
u_{max}	Maximum control input torque produced by the actuators [Nm]

1 Introduction

1.1 Motivation

Being able to accurately determine and control the attitude of a spacecraft is necessary for achieving any mission's goal. The Attitude Determination and Control System (ADCS) makes use of sensors, to determine the orientation and angular velocity of a spacecraft in its orbit, and, actuators to correct the position when necessary. However, the presence of not quantifiable external disturbances as well as system parameter uncertainty makes its design a challenging task. Furthermore, due to the non-linearity of the problem, classic linear control techniques can not be automatically applied and ensuring stability and robustness can be a problem. Similarly, the accuracy of the mathematical model of the dynamics and hardware highly influences attitude predictions.

1.2 Overview & objectives

This report will firstly detail the construction of the dynamic and kinematic equations of motion. These represent the dynamical model of the system and will be the base for the construction of the attitude prediction and control algorithm. A quaternion-based extended Kalman filter will then be designed for the mission and tested against a reference trajectory. Finally, a non-linear Proportional-Integral-Derivative (PID) controller based on Lyapunov stability will be constructed. To assess the efficiency of the controller, a reference attitude and angular velocity change is commanded. Furthermore, a specific configuration of Control Moment Gyroscopes (CMGs) will be considered and possible limitations will be analysed. An overview of the attitude determination and control system can be found in the block diagram in Figure (1).

2 System dynamical equations

Two systems of right-handed reference frames will be used in this report. The Inertial Reference Frame (**IRF**) is an earth centered equatorial reference frame. In this frame, the x axis points towards some inertial reference point and the z axis passes through the equatorial [3]. The second frame used will be the Body Reference Frame (**BRF**). This frame has its origin at the body's center of mass and its axes are in the direction of the principal moment of inertia. The attitude of the spacecraft

will be described using quaternions rather than Euler angles. Firstly, they don't suffer from singularity problems and secondly any rotation between frames can be expressed using one matrix, greatly reducing computational costs. Quaternions are constituted by an imaginary part and a real part such that

$$q = q_1 i + q_2 j + q_3 k + q_4 \quad (2.1)$$

Using the quaternion differential equation expressed in equation (2.2), it is possible to describe the kinematic of the spacecraft.

$$\dot{\mathbf{q}} = \frac{1}{2} \boldsymbol{\Omega} \mathbf{q} = \frac{1}{2} \begin{bmatrix} 0 & \omega_z & -\omega_y & \omega_x \\ -\omega_z & 0 & \omega_x & \omega_y \\ \omega_y & -\omega_x & 0 & \omega_z \\ -\omega_x & -\omega_y & -\omega_z & 0 \end{bmatrix} \begin{bmatrix} q_1 \\ q_2 \\ q_3 \\ q_4 \end{bmatrix} \quad (2.2)$$

The attitude dynamics of the spacecraft is governed by the Euler's equation of motion. This describes the rate of change of the body angular velocity with time and it is introduced in equation (2.3).

$$J\dot{\boldsymbol{\omega}} = \mathbf{T}_a + \mathbf{T}_d - (\boldsymbol{\omega} \times J\boldsymbol{\omega}) \quad (2.3)$$

External disturbances torques could include gravity gradient, solar pressure and magnetic torque [3]. The required actuator torques are created by thrusters and CMGs.

3 Extended Kalman Filter

3.1 Principle

The Kalman filter is a recursive, optimisation and data processing algorithm. This mathematical tool allows to estimate the state variables of a system using information from noisy measurements and from a model of the system which should be linear. For the spacecraft it was decided to implement an Extended Kalman Filter. In this case, linearisation of the nonlinearities present in the system's process model will be required at each time step, so as to obtain a discrete, time-varying system. The EKF algorithm can be divided in two steps: prediction and correction. [4]

1. During the prediction phase the equation of motions are initially linearised and an estimate of the state at the future time step (t_{k+1}) is obtained based on the system model and on the prior state estimate.

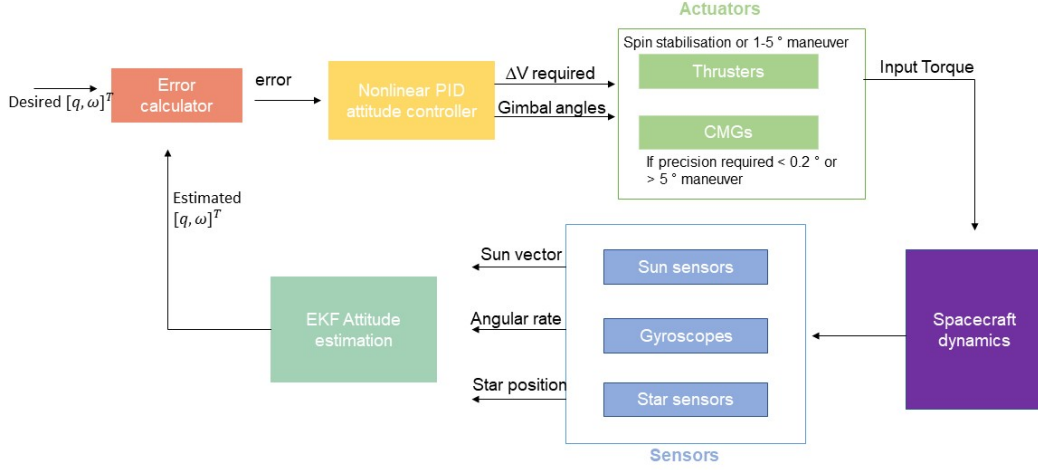


Figure 1: Attitude determination and control block diagram

2. During the correction phase the current measurements are used to correct the estimate of the state obtained during the prediction phase.

The state transition matrix approximates the change of the state in the current time step Δt [5] and it can be calculated using the matrix exponential as shown in equation (3.6).

3.2 Mathematical Process

3.2.1 Propagation Cycle

In order to design a 7 states Kalman Filter the state vector is given by

$$x = [q_1 \ q_2 \ q_3 \ q_4 \ \omega_x \ \omega_y \ \omega_z]^T \quad (3.1)$$

The continuous non linear model and measurement model are defined as

$$\dot{x} = f(x(t), u(t), t) + G(t)w(t) \quad (3.2)$$

$$y = h(x(t), t) + v(t) \quad (3.3)$$

In this model $w(t) \sim N(0, Q)$ and $v(t) \sim N(0, R)$ are zero-mean Gaussian noises. Q accounts for the state process noise and R considers the expected measurement noise. The discrete state space expression is represented in equation (3.4), where the Jacobian of f in (3.5) is evaluated at the previous time step state $(x_{(k|k)})$.

$$\dot{x}_{(k+1|k)} = F \cdot x_{(k|k)} + G \cdot u_{(k|k)} + w_{(k|k)} \quad (3.4)$$

$$F(k) = \left. \frac{\partial f}{\partial x} \right|_{x=x_{(k|k)}}, \quad G(k) = \left. \frac{\partial f}{\partial u} \right|_{u=u_{(k|k)}} \quad (3.5)$$

$$\Phi_{(k+1)} = e^{F(k)\Delta t} \approx I + A\Delta t + \dots \quad (3.6)$$

where I is the identity matrix. The *estimation error covariance matrix* (P) quantifies the error between estimated ($\hat{x}$) and real state (x). It can be defined as [6]

$$P = E[(x - \hat{x})(x - \hat{x})^T] \quad (3.7)$$

The P matrix, as the other state variables, needs to be propagated in time using the state transition matrix (3.8).

$$P_{(k+1|k)} = \Phi_{(k+1)} P_{(k|k)} \Phi_{(k+1)}^T + Q_{(k+1)} \quad (3.8)$$

On the other hand, the prediction of the state can be obtained solving the integral in (3.9) with a numerical method such as Runge-Kutta.

$$x_{(k+1|k)} = x_{(k|k)} + \int_{t_k}^{t_{k+1}} f(t) dt \quad (3.9)$$

3.2.2 Correction Cycle

In order to correct the initially predicted values, a measurement model is necessary. Starting from equation (3.3), the observation matrix is defined as

$$H(k) = \left. \frac{\partial h}{\partial x} \right|_{x=x_{(k|k)}} \quad (3.10)$$

The output of the system corresponds to the sensor measurements and hence depends on the type of sensor used. As explained in [7], a combination of sun sensors, gyroscopes and star sensors was chosen. The measurements of all the sensors need to be incorporated in the EKF algorithm. It was decided to use these measurements in a sub-sequential order as the computational cost is lower than a parallel approach and good accuracy is ensured [8]. With this approach the least accurate sensor (gyroscope) will be initially used and the Kalman filter algorithm will be run in order to obtain a first state estimate. This estimate will be the input value for future iterations using the measurements from the other two sensors. Since the sensors measurements are given in body frame it can be then inferred that [5]

$$H = \frac{\partial z_B}{\partial x} = \frac{\partial R_{bi} z_i}{\partial x} \quad (3.11)$$

where the rotational matrix from inertial to body frame can be easily expressed using quaternions.

$$R_{bi} = \begin{bmatrix} q_1^2 - q_2^2 - q_3^2 - q_4^2 & 2(q_1 q_2 + q_4 q_3) & 2(q_1 q_2 - q_4 q_3) \\ 2(q_1 q_2 - q_4 q_3) & -q_1^2 + q_2^2 - q_3^2 + q_4^2 & 2(q_2 q_3 + q_4 q_1) \\ 2(q_1 q_3 + q_4 q_2) & 2(q_2 q_3 - q_4 q_1) & -q_1^2 - q_2^2 + q_3^2 + q_4^2 \end{bmatrix} \quad (3.12)$$

Since R_{bi} in equation (3.12) depends solely on the attitude, H can be re-written as

$$H = \begin{bmatrix} \frac{\partial R_{bi}}{\partial q_1} z_i & \frac{\partial R_{bi}}{\partial q_2} z_i & \frac{\partial R_{bi}}{\partial q_3} z_i & \frac{\partial R_{bi}}{\partial q_4} z_i & 0_{3 \times 3} \end{bmatrix} \quad (3.13)$$

Now all the information necessary for computing the Kalman filter gain are obtained. The Kalman filter gain is obtained based on the value of the covariance matrices P , Q and R and it is the optimal gain in order to minimise the mean-square error. K is calculated with equations (3.14)-(3.15)

$$S_{(k+1)} = H P_{(k+1|k)} H^T + R_{(k+1)} \quad (3.14)$$

$$K_{(k+1)} = P_{(k+1|k)} H^T S_{(k+1)}^{-1} \quad (3.15)$$

Finally the states can be updated with (3.16) and (3.17).

$$x_{(k+1|k+1)} = x_{(k+1|k)} + K_{(k+1)} [z_B - R_{bi} z_i] \quad (3.16)$$

$$P_{(k+1|k+1)} = P_{(k+1|k)} - K_{(k+1)} H P_{(k+1|k)} \quad (3.17)$$

3.3 Simulation & Testing

3.3.1 Parameters initialisation and choices

In order to implement the EKF a number of parameters need to be initialised. The importance of this initialisation will be outlined in this section [9].

- **Initial state:** The initial state estimate influences the accuracy of the EKF. Since linearisation is required, a good guess of the initial estimate is crucial to obtain the desired performance.
- **Initial state covariance:** P_0 represents the accuracy of the first state estimate. The choice of P_0 should be consistent with the choice of the initial state estimate. As an example, if P_0 is chosen to be small whereas the error in the initial state guess is large, the Kalman filter gain will be small and the estimator will put too much weight on the model predictions [10].
- **Process noise covariance:** It is one of the hardest parameters to initialise [10]. To accurately estimate Q , random process noises, parameters and structural errors in the *Arceus* model need to be quantified.
- **Measurement noise covariance:** R estimates the noises entering the measurement model and it highly depends on the accuracy of the sensors chosen. It can be obtained from the measurement device's manufacturer.

The choice of these parameters for simulation will be explained in this section. A code for the EKF algorithm has been created on MATLAB and can be found in Appendix B. In order to test the EKF, the following initial angular velocity $\omega_0 = [0.5 \ 0.5 \ 0.5]^T$ (rad/secs) and attitude $q_0 = [1 \ 0 \ 0 \ 0]^T$ have been chosen. The equations of motions (2.2) and (2.3) are solved with the inbuilt ODE45 solver to simulate the actual trajectory over time. For simplicity, external disturbances and actuator torques are neglected and will be considered in the sections to follow. Furthermore, only one sensor measurement, the sun sensor, has been used to obtain the observation matrix (3.10). That is because a model prediction can be easily obtained for this type of sensor and good accuracy is ensured. Since a time span of 30 seconds has been implemented for the simulation, the sun vector in inertial frame can be assumed fixed in time (as only affected by the Julian date). Knowing the Julian Date, the number of centuries since Greenwich noon, Terrestrial Time, on 1 January 2000 (J2000) is calculated as

$$T = \frac{J - 2451545}{36525} \quad (3.18)$$

Then, the mean longitude of the sun (3.19) and the mean anomaly of the sun (3.20) are obtained with

$$\lambda_{sun} = 280.4606184 + 36000.77005361 \cdot T \quad (3.19)$$

$$M_{sun} = 357.5277233 + 3599905034 \cdot T \quad (3.20)$$

The ecliptic longitude of the sun and the obliquity of the ecliptic are then computed with (3.21) and (3.22).

$$\lambda_e = \lambda_{sun} + 1.914666471 \cdot \sin(M_{sun}) + 0.918994643 \cdot \sin(2M_{sun}) \quad (3.21)$$

$$e = 23.439291 - 0.0130042 \cdot T \quad (3.22)$$

Finally the sun vector in inertial frame can be calculated with (3.23).

$$s_i = \begin{bmatrix} \cos(\lambda_e) \\ \cos(e) \sin(\lambda_e) \\ \sin(e) \sin(\lambda_e) \end{bmatrix} \quad (3.23)$$

To simulate on board sensors measurements, which will be made in body frame, the sun vector s_i is rotated into body frame using the rotational matrix (3.12) such that $s_b = R_{bi} \cdot s_i$. A white Gaussian noise of mean zero and variance 0.2 is added to simulate inaccuracies in sensors measurements as suggested in [9], where 0.2 is considered the worst case scenario. To initialise the filter, a suitable initial state guess is needed. For simulations purpose, while the true initial state (x_0) remains unchanged, the initial state estimate ($\hat{x}_0$) is randomly generated. Assuming a certain knowledge of the state, it is decided that the following range of ω (3.24) and q (3.25) are acceptable guesses.

$$\hat{\omega}_0 \in [0.5, 1] \times [0.5, 1] \times [0.5, 1] \quad (3.24)$$

$$\hat{q}_0 \in [0.5, 1] \times [0, 0.1] \times [0, 0.1] \quad (3.25)$$

Using the recommendations presented in [10] and [6], the error covariance matrix is initialised as followed

$$P_0 = \text{diag}((\hat{x}_0 - x_0)(\hat{x}_0 - x_0)^T) \quad (3.26)$$

Finally, the Q and R matrices are obtained from knowledge of the expected measurements noises from sun sensor as well as random noises. These matrices were tuned and adjusted with the objective of minimising the error shown in Figure (11) between real and estimate state. They are introduced in (3.27) and (3.28) and they are assumed to be constant with time.

$$Q_0 = \begin{bmatrix} 0.008^2 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0.008^2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0.008^2 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0.008^2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0.008^2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0.008^2 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0.008^2 \end{bmatrix} \quad (3.27)$$

$$R_0 = \text{diag}((0.1^2, 0.1^2, 0.1^2)) \quad (3.28)$$

3.3.2 Results

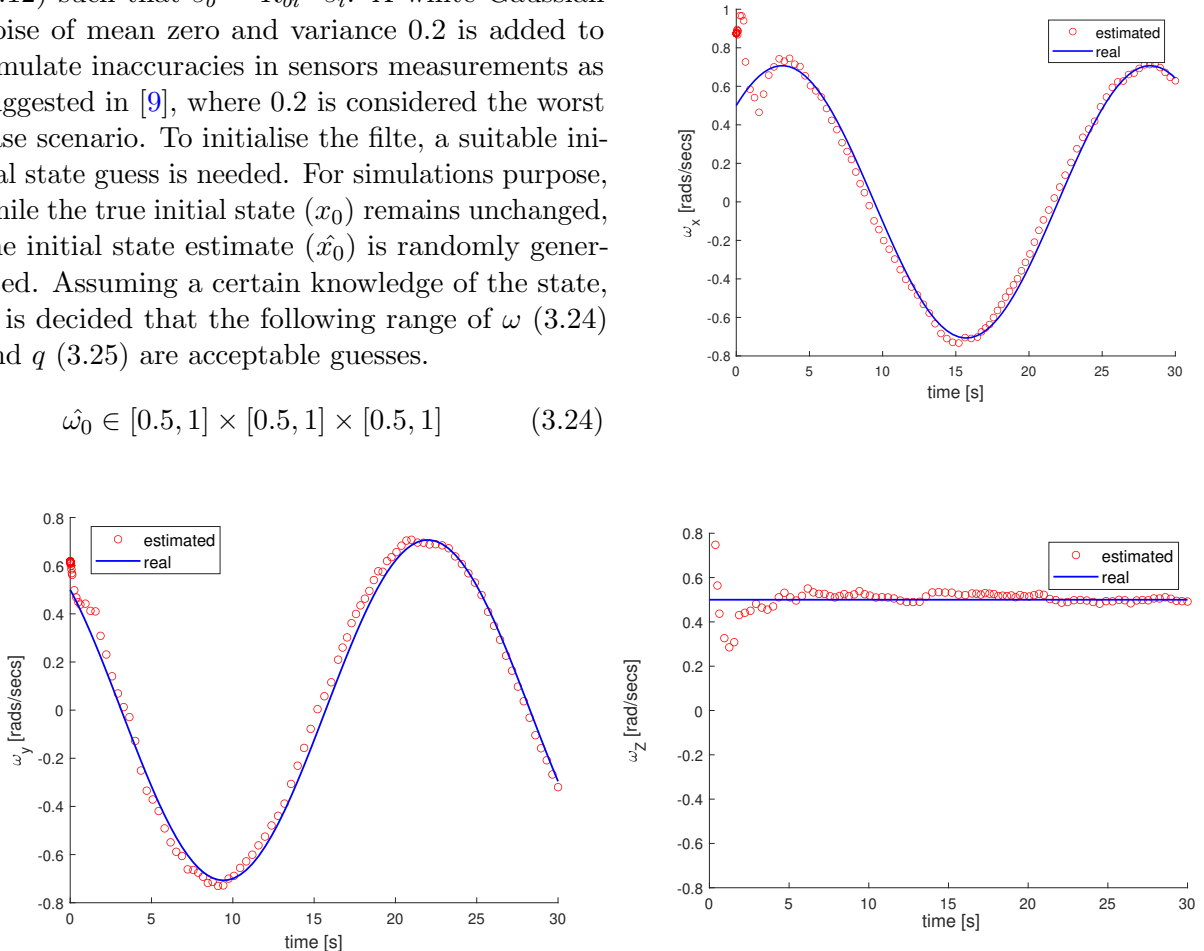


Figure 2: Comparison between real and estimate change of ω with time

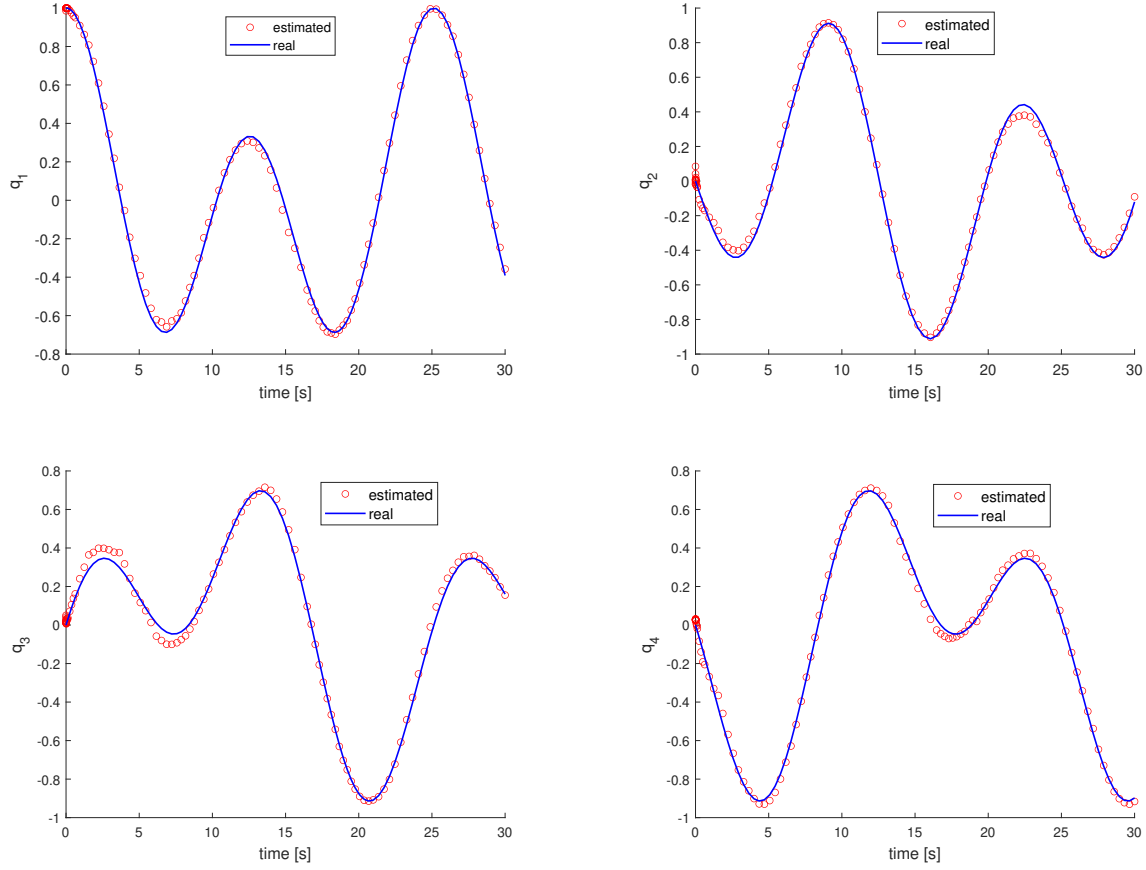


Figure 3: Comparison between real and estimate change of q with time

As presented in Figure (2), for this specific run and specific trajectory, the initial estimate conditions ($\hat{\omega}_0$) differ significantly from the real ones (ω_0). Nevertheless, the EKF algorithm is able to adjust its estimate until a satisfactory convergence is reached.

A similar conclusion can be inferred for the attitude behaviour. Figure (3) shows how the EKF is able to successfully predict the change in q with time.

To better assess the accuracy of the EKF, a comparison between the square of the error ($[\hat{x} - x][\hat{x} - x]^T$) and the covariance matrix error is necessary. In fact, the diagonal entries of the P matrix are expected to give an estimate of the square of the errors. For example, $P(1, 1) \approx (\hat{q}_1 - q_1)^2$. It should be noted that, since an EKF is designed and linearisation is required, really rarely the error will be exactly equal to the P matrix.

Table 1: Comparison between mean value of P matrix entries and error in q

q_1	$6.32 \cdot 10^{-4}$	q_2	$7.07 \cdot 10^{-4}$
$P(1, 1)$	$8.25 \cdot 10^{-4}$	$P(2, 2)$	$1.00 \cdot 10^{-3}$
q_3	$8.60 \cdot 10^{-4}$	q_4	$6.50 \cdot 10^{-4}$
$P(3, 3)$	$1.2 \cdot 10^{-3}$	$P(4, 4)$	$1.1 \cdot 10^{-3}$

Table 2: Comparison between mean value of P matrix entries and error in ω

ω_x	0.0298	ω_y	0.0039
$P(5, 5)$	0.0302	$P(6, 6)$	0.0060
ω_z	0.0416		
$P(7, 7)$	0.0442		

Tables (1) and (2) show a comparison between the mean variance and mean value of the diagonal entries of P during the time frame of the simulation. The results are in good agreement, with P slightly overestimating the error. A graph showing the error trend with time is reported in Figure (11) in Appendix A.

4 Attitude Control

4.1 Introduction to Nonlinear control techniques

It has been shown in Section (2) that the equations governing the change in attitude with time are nonlinear. Furthermore, unexpected and unmodeled perturbations can occur. If a linear control technique was to be implemented, Jacobian linearisation would be required. However, stability issues can arise and only local stability can be ensured [11]. Hence, nonlinear control techniques are explored. This will ensure robustness, asymptotic control [12] and higher precision which is necessary for a large-scale, crewed mission. Two types of control techniques are usually implemented when it comes to spacecraft attitude control. Those are introduced in [12] and [13] and will be briefly summarised here.

4.1.1 Lyapunov control law

Defining the state vector $e = x - x_r$ as the difference between the reference state and the actual one, the closed loop dynamical equation of motion can also be expressed as

$$\dot{e} = f(e, u(t)) \quad (4.1)$$

In order to analyse the stability of the system, a scalar energy-like function needs to be identified. This is called Lyapunov function and it is expressed as $V = V(e)$. To ensure stability, $V(e)$ needs to satisfy a set of conditions, namely;

$$V(0) = 0 \quad (4.2a)$$

$$V(e) > 0 \text{ for } e \neq 0 \quad (4.2b)$$

$$V(e) \rightarrow \infty \text{ for } \|e\| \rightarrow \infty \quad (4.2c)$$

$$\dot{V}(e) < 0 \text{ for } e \neq 0 \quad (4.2d)$$

A control input $u(t)$ which is based on the Lyapunov function can then be identified.

4.1.2 Feedback linearisation

In this case, a change of variables is required such that the non linear system can be transformed into the standard feedback linearisation form of (4.3).

$$\dot{x} = Ax + B\beta^{-1}(x)[u - \alpha(x)] \quad (4.3)$$

By choosing a control input of the type of (4.4), the system in (4.3) is transformed into the linear system (4.5).

$$u = \alpha(x) + \beta(x)\nu \quad (4.4)$$

$$\dot{x} = Ax + B\nu \quad (4.5)$$

4.2 Nonlinear PID controller design

4.2.1 Controller selection

Even though the control input form differs depending on the method selected, there is interconnection between the Lyapunov control law and feedback linearisation. It is in fact possible to identify a Lyapunov function in both cases. With the latter method, the reference trajectory is predefined which can be advantageous, however, usually the kinematic equation of motion is not taken into account and singularities can occur when β in (4.3) goes to zero. Although techniques to avoid singularities are explored ([14] et al.), it was decided that, for the purpose of the mission and for completeness, a Lyapunov control law approach would be more appropriate.

4.2.2 Controller design

The objective of the controller should be to track a reference attitude and angular velocity as well as to stabilise the rotation of the *Arceus*.

For the generic case of attitude and angular velocity control, a Lyapunov function which contains both the error in attitude quaternion as well as in angular velocity is required. Only error in the first three quaternions is tracked as q_{4e} is constrained by

$$q_{1e}^2 + q_{2e}^2 + q_{3e}^2 + q_{4e}^2 = 1 \quad (4.6)$$

Based on the work of [15] and [13] a good candidate of Lyapunov function is expressed in equation (4.7).

$$V(\epsilon, \delta\omega) = \frac{1}{2}\delta\omega^T J \delta\omega + K\epsilon^T \epsilon \quad (4.7)$$

Taking the derivative of (4.7) and making uses of (2.3), equation (4.8) is obtained.

$$\dot{V} = \delta\omega^T \left(J \frac{{}^B d}{dt} \delta\omega + K\epsilon \right) \quad (4.8)$$

Where B means that the derivative of $\delta\omega$ is taken as seen in the body frame. However, ω_r is not necessarily given in body frame but rather in a generic reference frame R . It can be inferred that

$$\frac{{}^B d}{dt} \delta\omega = \dot{\omega} - \dot{\omega}_r + \omega \times \omega_r \quad (4.9)$$

To ensure stability, as stated in (4.2), $\dot{V}$ should be negative and semi-defined. Hence by setting

$$\dot{V} = -\delta\omega^T P \delta\omega \quad (4.10)$$

and comparing (4.10) with (4.8), the stability constraint in (4.11) is obtained. Another requirement is that P must be a positive matrix.

$$J \frac{Bd}{dt} \delta\omega + K\epsilon + P\delta\omega = 0 \quad (4.11)$$

Making use of (4.9) and substituting (2.3) in (4.11), an expression for the feedback, globally stabilising, control torque required is found in (4.12).

$$u = -K\epsilon - P\delta\omega + J\dot{\omega}_r - \omega \times \omega_r + \omega \times J\omega - T_d \quad (4.12)$$

where $u(t)$ corresponds to T_a in (2.3).

4.2.3 Unmodeled disturbances & Saturation

If some unmodeled external disturbances are presented in the system then T_d in (2.3) can be expressed as $T_d = u + \Delta T$. If the same control law of (4.12) is kept, the Lyapunov derivative assumes the form of (4.13).

$$\dot{V} = -\delta\omega^T P \delta\omega + \delta\omega^T \Delta T \quad (4.13)$$

$\dot{V}$ is not anymore negative and semi-defined for $\Delta T \neq 0$ and, thus, the control law is not globally stabilising. The attitude error will reach a steady state error equal to

$$\epsilon_{SS} = \lim_{t \rightarrow \infty} \epsilon = \frac{1}{K} \Delta T \quad (4.14)$$

To achieve asymptotic tracking, a new state variable representing the integral of the error is therefore introduced in (4.15).

$$z(t) = \int_0^t (K\epsilon + J\delta\dot{\omega}) dt \quad (4.15)$$

Adopting a similar approach as in section 4.2.2, the new control law which ensures global stability is found to be

$$u = -K\epsilon - P\delta\omega - PK_i z + J\dot{\omega}_r - \omega \times \omega_r + \omega \times J\omega - T_d \quad (4.16)$$

This form of the controller will be used for simulations since unmodeled disturbances are expected. In an ideal case there is not an upper bound in the amount of control input torque produced by the

actuators. However, this is not always the case. To account for saturation issues, the control law in (4.16) can be modified to obtain (4.17).

$$u(t) = \begin{cases} u & \text{for } |u| < u_{max} \\ u_{max} \cdot \text{sgn}(u) & \text{for } |u| > u_{max} \end{cases} \quad (4.17)$$

where $\text{sgn}(u)$ represents the sign function and it is equal to +1 when u is positive and -1 when u is negative.

4.3 Simulation & Testing

The response of the *Arceus* to a typical maneuver has been simulated and tested on MATLAB. The maneuver consists in both a attitude and angular velocity reference tracking. This is expected to be required during the mission life. The parameters chosen for simulations have been listed in Table (3).

Table 3: Parameters chosen for controller simulation

Parameter	Value	Unit
ϵ_0	$[0.2, 0.4, 0.5]^T$	[-]
ω_0	$[0.02, 0.01, 0.03]^T$	$[\frac{\text{rad}}{\text{secs}}]$
ω_r	$[0, 0, 0]^T$	$[\frac{\text{rad}}{\text{secs}}]$
T_d	$[14, 14, 14]^T$	[Nm]
ΔT	$[5, 5, 5]^T$	[Nm]
K	$20 \cdot 10^2$	$[\frac{\text{kgm}^2}{\text{secs}^2}]$
P	$([60 \cdot 10^3, 60 \cdot 10^3, 150 \cdot 10^3])$	$[\frac{\text{kgm}^2}{\text{secs}}]$
K_i	$([3, 3, 3])$	$[\text{secs}^{-1}]$
$ u _{max}$	2000	[Nm]

The initial attitude error is deliberately chosen to be large and the spacecraft is required to be at rest at the end of the maneuver. A 14 Nm disturbance torque is added on each axis, in line with what it is expected to be the worst case scenario during the mission [16]. Furthermore, a unmodeled external disturbance torque of 5 Nm is added to the model. This value is much higher than what it is usually experienced in space as T_d is already taken to be a conservative value. Nevertheless, the presence of K_i eliminates the steady state error. The gain matrices, where K is a scalar and P and K_i are diagonal matrices, are tuned by trial and error. The aim was to achieve a fast enough response while avoiding excessive oscillations since those would impart higher accelerations on the structure

and hence would cause additional stresses. Additionally, an exaggerated fast response could be badly received by the astronauts on board. Two set of simulations have been done. The first one considers an unconstrained control law with no limits in u_{max} and a controller of the type (4.16) is implemented. On the other hand, in the second simulation, it is assumed that a limit torque of 2000 Nm can be applied on each axis. Results are represented in Figures (4), (5), (6) and (7).

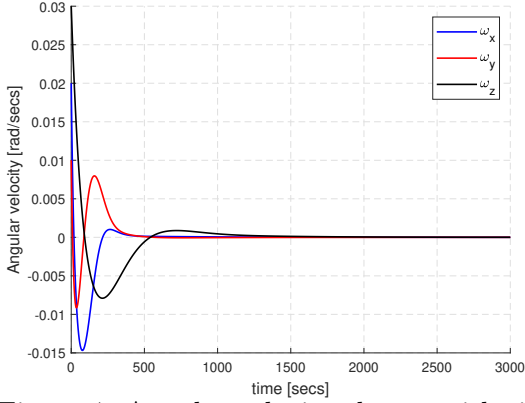


Figure 4: Angular velocity change with time

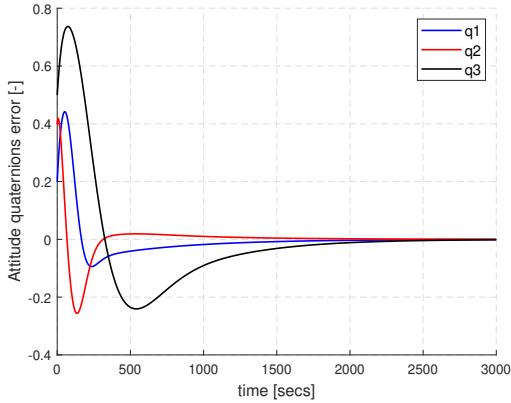


Figure 5: Attitude quaternion error change with time

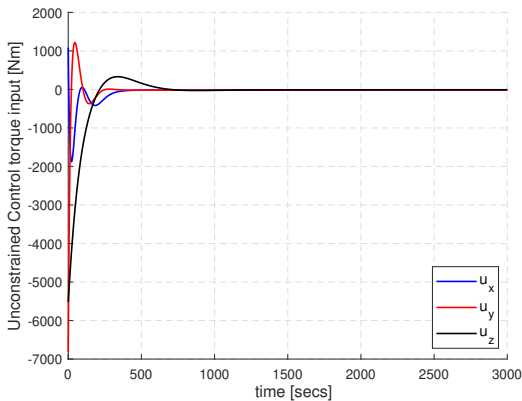


Figure 6: Control input torque with time when an unconstrained control law is implemented

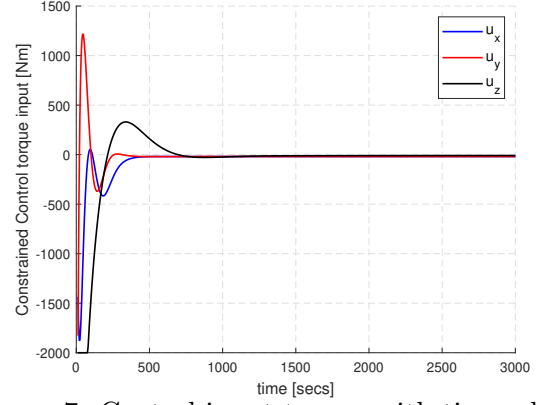


Figure 7: Control input torque with time when a constrained control law is implemented

As shown in Figure (4) and (5), steady state is reached after roughly 1500s and 2500s, respectively. The attitude error is more susceptible to external disturbances, but the integral term eventually brings the error to zero. When a constrained control law is applied, the control input in the z direction is saturated for the first 80s of the simulation.

5 Control Moment Gyroscopes (CMGs)

5.1 Actuators configuration

5.1.1 Overview

Figure (6) and (7) in Section (4.3) depicted the required actuators' control input for a typical maneuver. It should be clear that, even with the introduction of a saturation block, the torque that needs to be produced by the actuators is particularly high. This is to be expected due to the large scale of the vehicle. For the selection and sizing of actuators the reader is referred to [7]. The main difficulties arisen in the selection process will be outlined in this section.

CMGs are defined as momentum exchange devices. They create a torque by rotating the CMGs angular momentum vector around a secondary axis. By an exchange of angular momentum between the CMGs and the spacecraft, attitude control is achieved [2]. Furthermore, CMGs present a greater accuracy and precision compared to other actuator technologies.

However, the presence of a rotating and a stationary module on the *Arceus* makes the implementation of this technique more complicated and less practical in some instances. Thus it was decided

to use:

1. **Thrusters** for:

- Spin stabilisation
- Attitude maneuvers of less than 5° where small oscillations are considered acceptable

2. **CMGs** for:

- Attitude maneuvers of more than 5°
- Attitude maneuvers where an error of less than 0.2° is required

The sizing and architecture of thrusters can be found in [17]. For those maneuvers where CMGs are needed, the ring structure will de-spin before the maneuver is started.

5.1.2 CMGs configuration

The momentum envelope potential of the system is dependent on the configuration of CMGs chosen [2]. Due to the high control input required, 4 CMGs placed in a pyramid configuration are implemented as shown in Figure (8).

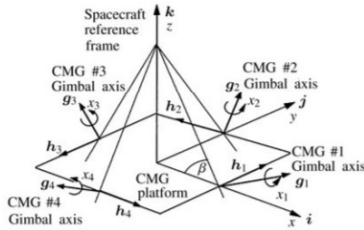


Figure 8: CMGs placed in a pyramid configuration [1]

In this configuration, the momentum envelope assumes a spherical shape which means that an equal amount of control torque can be applied in each of the three axis. Hence, rapid attitude maneuver can be achieved [1]. Furthermore, even if one of the CMGs is in a singular state, control in all three directions can still be obtained [2]. Nevertheless, singularity is an issue that needs to be addressed when designing CMGs. Singularity occurs when two or more CMGs have the momentum vector aligned or pointed in opposite direction, such that a degree of freedom is lost. Several techniques have been developed to counteract singularities as it will be discussed in Section (5.2.2). Finally, when the maximum angular momentum state is approached, the system is said to be saturated and de-saturation is required.

5.2 CMGs mathematical model

5.2.1 Steering law

Starting from the Euler equation of motion (2.3), making use of the conservation of total angular momentum, the actuator torque can be linked to the angular momentum stored in the CMGs by equation (5.1).

$$\dot{h}_{cmg} + \omega \times h_{cmg} = -T_a \quad (5.1)$$

where, in this case, T_a is the torque required for a given attitude maneuver and it can be found with equation (4.16) or (4.17). In addition, the angular momentum of the CMGs is a function of the gimbal angles $g = [g_1, g_2, g_3, g_4]$ and the skew angle β , both are indicated in Figure (8), [2]. For a pyramid configuration, the aforementioned relationship is outlined by Bong Wie [18].

$$h = \sum_{n=1}^4 h_n(g_n) = h_1 \begin{bmatrix} -c\beta \sin g_1 \\ \cos g_1 \\ s\beta \sin g_1 \end{bmatrix} + h_2 \begin{bmatrix} -\cos g_2 \\ -c\beta \sin g_2 \\ s\beta \sin g_2 \end{bmatrix} + h_3 \begin{bmatrix} c\beta \sin g_3 \\ -\cos g_3 \\ s\beta \sin g_3 \end{bmatrix} + h_4 \begin{bmatrix} \cos g_4 \\ c\beta \sin g_4 \\ s\beta \sin g_4 \end{bmatrix} \quad (5.2)$$

where $s\beta$ and $c\beta$ are abbreviation of $\sin \beta$ and $\cos \beta$. Now, for a pyramid configuration, the skew angle is fixed and equal to $\beta = 54.73^\circ$. Moreover, the different CMGs have the same angular velocity and angular momentum capability so that $h_1 = h_2 = h_3 = h_4 = h_0$. Taking the time derivative of (5.2) the set of relationships shown in (5.3), (5.4) and (5.5) are obtained.

$$\dot{h}_{cmg} = A(g)\dot{g} \quad (5.3)$$

$$A = \frac{\partial h}{\partial g} \quad (5.4)$$

$$A = h_0 \begin{bmatrix} -c\beta \cos g_1 & \sin g_2 & c\beta \cos g_3 & -\sin g_4 \\ -\sin g_1 & -c\beta \cos g_2 & \sin g_3 & c\beta \cos g_4 \\ s\beta \cos g_1 & s\beta \cos g_2 & s\beta \cos g_3 & s\beta \cos g_4 \end{bmatrix} \quad (5.5)$$

Knowledge of the commanded $\dot{h}_{cmg}$ for an attitude maneuver allows to obtain the required gimbal rate angle $\dot{g}$ with equation (5.3). Since the matrix $[A]$ is singular, a direct inverse can not be computed. Therefore, most of the literature ([18], [2] et al.) takes the Moore Penrose pseudo-inverse of A (5.6).

$$\dot{g} = A^T(AA^T)^{-1}\dot{h}_{cmg} \quad (5.6)$$

Table 4: Comparison between singularity avoidance methods as presented in [2]

	Method	Induced Error	Momentum Envelope Efficiency	Covergence towards Singularity	Capabilities for System Changes
Offline Planning	Optimised pre-modelling	None	Slightly reduced	Impossible	Extensive computational power
Preferred Gimbal Angle	Optimised initial gimbal orientations	None	Slightly reduced	Impossible	Extensive computational power
Linearly Constrained	Restricted relative gimbal angles	None	Reduced	Impossible	Fully adaptable
Null Motion	Randomly induced torque scaling when singularity approached	Large near singularity	Full use + large induced error	Possible	Fully adaptable
Singularity Robust Inverse	Optimised null motion scaling	Controller near singularity	Full use	Possible	Fully adaptable

5.2.2 Singularity avoidance

In any type of steering law and in any configuration of CMGs, singularities can be expected and need to be accounted for. From equation (5.5), it can be shown that when $rank(A) < 3$ the determinant of (AA^T) is zero and the system goes through a singular state. In fact, it has been said that the individual CMGs impart an equal angular momentum change to the overall system, thus a singular state is inevitably approached where $h_i = \pm h_j$ with $i, j = 1, 2, 4$ and $i \neq j$.

Many modifications of the steering law in (5.6) have been proposed so as to avoid singular states. Those are commonly divided in singularity-avoidance techniques and singularity-robust laws. While with singularity-avoidance convergence towards a singular state is impossible, with singularity-robust laws the system is allowed to enter a singular state, in those instances where the singularity can be safely controlled. A full list of singularity avoidance laws can be found in [2]. A summary is presented in Table (4). For the *Arceus*, a Singularity Robust Inverse approach was implemented. Mainly because, due to the large torque required for maneuvers, a full momentum capability is required. Secondly, it is not desirable to adopt a technique which requires extensive computational power. Therefore, the steering law (5.6) is modified into equation (5.7)

$$\dot{g} = A^T(AA^T + \rho I)^{-1}\dot{h}_{cmg} \quad (5.7)$$

where I is the identity matrix and ρ is a scaling factor. The value of which has been obtained experimentally through optimisation by Yoon and Tsotras [19]. Calling $M = \sqrt{\det(AA^T)}$, M gives an indication of how close the system is to a singular state, then, according to [19], the scaling factor is chosen depending on the value of M (5.8).

$$\rho = 0 \text{ when } M > 1 \quad (5.8a)$$

$$\rho = \frac{0.1}{M} \text{ when } \frac{0.1}{M} < 0.2 \quad (5.8b)$$

$$\rho = 0.2 \text{ otherwise} \quad (5.8c)$$

5.3 Simulation & Testing

The same attitude maneuver simulated in section (4.3) will be considered in this section. Two sets of simulation have been run. In the first one, the unconstrained control input of Figure (6) has been considered, whereas, in the second one, the control law showed in Figure (7) will be used.

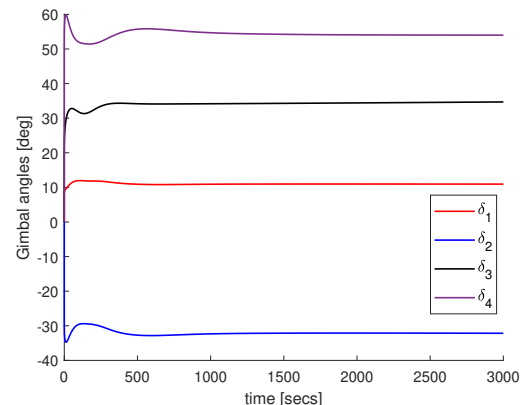


Figure 9: Gimbal angles required for the maneuver in an unconstrained case

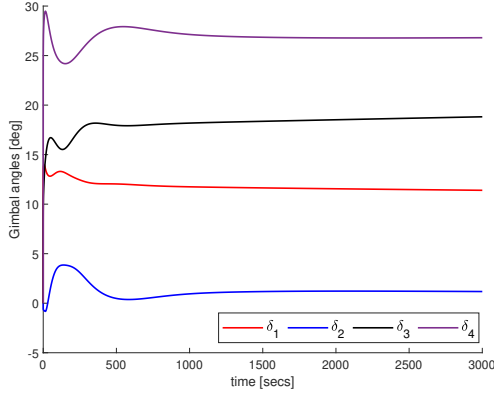


Figure 10: Gimbal angles required for the maneuver in a constrained case

Knowing the actuators control torque required, equation (5.1) allows to obtain the angular momentum and rate of change of angular momentum needed. From there, the modified steering law (5.7) is used to compute the gimbal angles required. For the purpose of the simulation, the initial gimbal angles has been set equal to $g_0 = [0, 0, 0, 0]$ deg and the initial angular momentum stored in the CMGs is $h_{cmg_0} = [0, 0, 0]^T$ Nms. The result are depicted in Figure (9) and (10) where it can be seen how singular states are avoided and the gimbal angles reach a steady state value when the maneuver is completed.

5.4 Failure analysis

At this stage, a complete failure analysis has not been done for the system. However, a general overview of the different failure possibilities will be given in this section and it is mainly based on the work of [1], [20] and [21]. The consequences of an eventual system failure are highly influenced by the number and the position of the CMGs that stop working. As a general rule, in case of failure the gimbal angle of the concerned CMGs should be kept fixed and its angular velocity should be reduced to zero. The following failure modes should be considered.

1. **One CMG failure:** The three CMGs remaining are sufficient to generate torque in all three axis. In principle, attitude maneuver are still possible. However, the lost symmetry in the system architecture adds complexity. Due to asymmetry and unchanged skew angle, the system has a non-zero initial angular momentum. To account for this, predefined initial non-zero gimbal angles can be set before starting the required maneuver.

2. **Two opposite CMGs failure:** In this instance symmetry in the system is regained and attitude maneuvers can be effectuated. However, the momentum envelope capability is greatly reduced and thrusters will be required to compensate for it.
3. **Two adjacent CMGs failure:** This failure mode consists in one of the worst case scenario. Because of the loss in symmetry and because of the reduced number of actuators working, maneuvers will be difficult to complete and the benefits of implementing CMGs as actuators will be lost. Hence, in those circumstances, it is desirable to use only thrusters for attitude maneuver.
4. **Three CMGs failure:** Even though control is complicated with only one gyro functioning, the remaining actuator can be used as a bias momentum wheel thanks to the ability of CMGs to store a large amount of angular momentum.

6 Discussion

In order to meet the mission requirements, an Attitude Determination and Control system for a large crewed vehicle has been designed and simulated. The main aim of this work was to achieve satisfactory accuracy while avoiding excessive computational cost. A seven states Extended Kalman Filter has been designed and tested. Even when only one sensor is used, the error in attitude didn't exceed 0.07 while the error in angular velocity was kept below 0.25 rad/secs. Furthermore, a control law based on Lyapunov stability has been realised. Numerical simulations were carried out in order to investigate the convergence of the system error and the robustness of the controller. An integral term has been incorporated so as to compensate constant unmodelled torques. However, the presence of external disturbances was not found to affect torque requirements significantly. These were mainly dictated by the high moment of Inertia of the *Arceus*. Finally, the Control Moment Gyros control law has been studied and validated. Singularities were avoided by the introduction of a correction term, thus, gimbal angles requirements were found. All the results were in line with what is expected for a mission of this scale.

7 Outlook

The work presented in this report is to be intended as a preliminary analysis. In order to have a better understanding of the accuracy of the ADCS, more simulations are required. The method by which sensors measurements are recreated could be improved by creating sensors models on Simscape, Simulink. This would allow a better simulation of the hardware configuration. A Monte-Carlo simulation of the EKF is also required before implementing the algorithm on board of the vehicle. Additionally, since gyroscopes will be used and they are known to be less accurate than other sensors, the effect of using gyroscopes on the accuracy of the EKF should be included. This is to check how much reliability should be placed in that specific sensor's measurements.

When it comes to the controller, even though the gains selected give satisfactory results, an optimisation of the gain, according to the mission needs, could be carried out, eventually implementing time varying gains. Finally, a modification of the steering law model, in case of CMGs failure, should be explored.

References

- [1] Meng T, Matunaga S. Failure-tolerant control for small agile satellites using single-gimbal control moment gyros and magnetic torquers. *Acta mechanica Sinica*. 2012;28(2):551–558. Available from: <http://lib.cqvip.com/qk/86601X/201202/42041408.html>.
- [2] McNamara N. Comparison of control moment gyroscope control laws for a spacesuit; 2017. Available from: <https://trove.nla.gov.au/work/228743454>.
- [3] Babcock E. CubeSat Attitude Determination via Kalman Filtering of Magnetometer and Solar Cell Data; 2011. Available from: https://explore.openaire.eu/search/publication?articleId=od_____1459::5ad40fdcd3dba47e816a4c908a3a9e3f.
- [4] Tuthill JD. Design and simulation of a nano-satellite attitude determination system; 2009. Available from: <http://hdl.handle.net/10945/4330>.
- [5] Roubache R, Benyettou M, Simohammed AM, Boudjemai A, Bellar A. Extended Kalman filter for attitude determination of an elliptical orbit satellites; June 2013. p. 291–296.
- [6] Simon D. Optimal State Estimation: Kalman, H Infinity, and Nonlinear Approaches. Wiley; 2006. Available from: <http://cds.cern.ch/record/991521>.
- [7] Chen S. MTV04-Subsystems, Attitude Determination and Control System Design Orbit Determination of Mars Transit Vehicle. Imperial College London, Department of Aeronautics; 2021.
- [8] Willner D, Chang CB, Dunn KP. Kalman filter algorithms for a multi-sensor system; December 1976. p. 570–574.
- [9] Unhelkar VV. Satellite Attitude Estimation using Sun Sensors, Horizon Sensors and Gyros ; 2012.
- [10] Schneider R, Georgakis C. How To NOT Make the Extended Kalman Filter Fail. *Journal of mechanical science and technology*. February 25, 2013;52:3354–3362.
- [11] Subbarao K. Nonlinear PID-Like Controllers for Rigid-Body Attitude Stabilization. *The Journal of the astronautical sciences*. 2004 Mar;52(1-2):61–74. Available from: <https://search.proquest.com/docview/2436893562>.
- [12] Skelton CEI. Mixed Control Moment Gyro and Momentum Wheel Attitude Control Strategies;. Available from: <http://hdl.handle.net/10919/9677>.
- [13] Junkins JL, Schaub H. Analytical Mechanics of Space Systems, Third Edition. Washington, DC: American Institute of Aeronautics and Astronautics, Inc; 2014. Available from: <http://arc.aiaa.org/doi/abs/10.2514/4.102400>.
- [14] Bang H, Lee JS, Eun YJ. Nonlinear Attitude Control for a Rigid Spacecraft by Feedback Linearization. *Journal of mechanical science and technology*. 2004;18(2):203–210. Available from: <https://www.dbpia.co.kr/journal/articleDetail?nodeId=NODE00531804>.
- [15] Leve FA, Hamilton BJ, Peck MA. Spacecraft momentum control systems. vol. 34. Cham: Springer; 2015. Available from: [https://ebookcentral.proquest.com/lib/\[SITE_ID\]/detail.action?docID=4178494](https://ebookcentral.proquest.com/lib/[SITE_ID]/detail.action?docID=4178494).
- [16] Wanot M. MTV04-Subsystems, Guidance and Navigation, and External Torque Analysis for Mars Transit Vehicle. Imperial College London, Department of Aeronautics; 2021.
- [17] Bivolaru M. MTV04-Subsystems, Thermal Control System and Nuclear Reactor Design. Imperial College London, Department of Aeronautics; 2021.
- [18] Wie B. Space vehicle dynamics and control. Reston, Va: AIAA; 1998.

- [19] Yoon H, Tsiotras P. Spacecraft Adaptive Attitude and Power Tracking with Variable Speed Control Moment Gyroscopes. *Journal of guidance, control, and dynamics*. 2002 Nov;25(6):1081–1090. Available from: <http://arc.aiaa.org/doi/full/10.2514/2.4987>.
- [20] Steyn W, Lappas V, Underwood C. Control moment gyro gimbal angle compensation using magnetic control during external disturbances. Reston: American Institute of Aeronautics and Astronautics; Jan 1, 2001. Available from: <https://search.proquest.com/docview/1431176121>.
- [21] Zhang J, Rachid A, Zhang Y. Attitude control for part actuator failure of agile small satellite. *Acta mechanica Sinica*. 2008;24(4):463–468. Available from: <http://lib.cqvip.com/qk/86601X/20084/28128002.html>.

Appendices

A Supplementary Plots

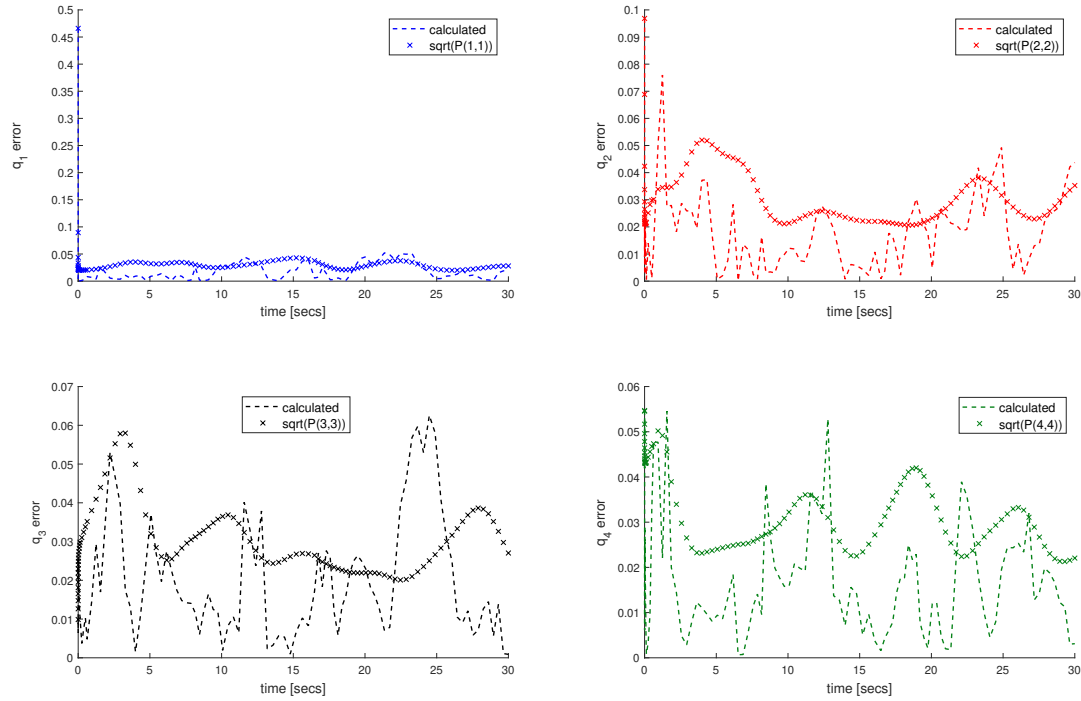


Figure 11: Comparison between attitude error over time and square root of diagonal entries of P matrix

B MATLAB Codes

This supplementary section will give a summary of some of the MATLAB Codes used for producing the results shown in the report. A brief introduction to each of them will be given. It should be noted that codes are shortened, when necessary, so that only the crucial features are presented.

B.1 EKF function

This function has been created in order to update the value of the states, as well as error covariance matrix, at each time step. It will be used in the main script to produce the EKF results.

```

1 function [x_new,P_new] = EKF(q, w_b, q_dot, w_b_dot, q_eq, w_eq, T_a,
    T_a_eq, dt, P, Q, R, R_bi, z_b, z_i )
2
3 %% Extended Kalman filter design and State Space
4 % Maria Luisa Scarpa – GDP MTV04
5
6 % Inputs required:
7 % q – attitude expressed in quaternion form body rate wrt inertial frame
8 % J – body moment of inertia
9 % w_b – angular velocity of the body frame relative to the inertial frame
10 % q_dot/ w_b_dot time derivatives of the above
11 % q_eq – result from attitude determination at previous timestep
12 % w_eq – as above
13
14
15 % Matrices initialisation:
16 % P – estimate error covariance matrix
17 % Q – process noise covariance matrix
18 % R – measurement noise covariance matrix
19 % dt – time step for discretisation (Seconds)
20 % https://apps.dtic.mil/dtic/tr/fulltext/u2/a430928.pdf
21
22
23 % Part A – propagation cycle
24
25 % creation of a state space model
26
27 x = [q; w_b];
28 f = [q_dot; w_b_dot];
29 u = T_a;
30 % Jacobian linearisation around equilibrium point
31 % x_dot = Ax + Bu
32
33 A = jacobian(f,x);
34 B = jacobian(f,u);
35
36 A = double(subs(A, [q; w_b], [q_eq; w_eq]));
37 B = double(subs(B, u, T_a_eq));
38 % Phi – state transition matrix – matrix exponential
39
40 Phi = expm(A*dt);
41

```

```

42 % Propagation state variables and covariance— Derivative for
    discretisation
43 % is approximated using Euler approximation.
44 % dw/dt = w(t_k) - w(t_k-1)/timestep
45
46 x_new_m = Phi*[q_eq;w_eq] + B*dt*T_a_eq;
47 P_new_m = Phi*P*Phi' + Q;
48
49
50 % Part B— Correction cycle 7 dimensions EKF
51 % Note that because two sensors measurements are used the Kalman filter
52 % will first use the first sensor to give a corrected state estimate and
53 % the second sensor measurement to upload this estimate
54
55 % z — Measurements of system state, either sun-sensor or magnetometer
56 % z_B — Body referenced measurements, directly from onboard sensors
57 % z_i — Inertial referenced measurements, from model prediction (IGRF) —
58 % separate function as it depends on the type of sensor
59
60 % H = dz_B/dx = dR_bi/dx * z_i
61 A1 = diff(R_bi, q(1));
62 A2 = diff(R_bi, q(2));
63 A3 = diff(R_bi, q(3));
64 A4 = diff(R_bi, q(4));
65
66 H = [A1*z_i A2*z_i A3*z_i A4*z_i zeros(3)];
67 H_m = double(subs(H, q, q_eq));
68
69 % K — Kalman Gain
70 K = P_new_m*H_m'*inv(H_m*P_new_m*H_m' + R);
71
72
73 % update state
74
75 A_rot = double(subs(R_bi, q, x_new_m(1:4)));
76
77 x_new = x_new_m + K*(z_b - A_rot*z_i);
78
79 P_new = P_new_m - K*H_m*P_new_m;
80
81 % normalisation of the quaternion for consistency
82
83 x_new(1:4) = x_new(1:4)/norm(x_new(1:4));
84
85
86 end

```

B.2 ODE solver

This script has been realised in order to solve the dynamic and kinematic equations of motion numerically. Furthermore, the sun vector in body frame is obtained.

```

1 % Simulate response of the spacecraft over time using ODE45 solver from
2 % some initial condition arbitrarily chosen.

```



```

3 % ADC & GNC – Maria Luisa Scarpa
4
5 %% solving for angular velocity
6 clear; clc
7
8
9 tspan = [0 30];
10 x0 = [0.5; 0.5; 0.5; 1; 0; 0; 0; 0];
11
12
13 [t, x] = ode45(@qw, tspan, x0);
14
15
16
17 %% Plots
18
19 % angular velocity with time
20 figure
21 hold on
22 plot(t, x(:,1), '.-k');
23 plot(t, x(:,2), '.-b');
24 plot(t, x(:,3), '.-r');
25 title('Angular Velocity with time');
26 legend('\omega_x', '\omega_y', '\omega_z');
27 xlabel('time [s]');
28 ylabel('angular velocity body wrt inertial [rad/ secs]');
29
30 % attitude with time
31 figure
32 hold on
33 plot(t, x(:,4), '.-k');
34 plot(t, x(:,5), '.-b');
35 plot(t, x(:,6), '.-r');
36 plot(t, x(:,7), '.-');
37
38 title('Quaternion attitude with time');
39 legend('q1', 'q2', 'q3', 'q4');
40 xlabel('time [s]');
41 ylabel('quaternions attitude bi');
42
43
44 %% Get sun vector in body frame at each time step
45
46 % because of the small time frame sun vector is assumed constant in
47 % inertial frame
48
49 si = sunpositionECI('19-May-2021');
50
51 for i = 1:length(t)
52     q = x(i,4:7);
53
54     R_bi = [q(1)^2-q(2)^2-q(3)^2+q(4)^2, 2*(q(1)*q(2) + q(3)*q(4)), 2*(q
              (1)*q(3) ...

```

```

55     - q(2)*q(4)); 2*(q(1)*q(2) - q(3)*q(4)), -q(1)^2 + q(2)^2 - q(3)^2 +
56     ...
57     q(4)^2, 2*(q(2)*q(3) + q(2)*q(4)); 2*(q(1)*q(3) + q(2)*q(4)), 2*q(2)*
58     q(3) ...
59     - q(1)*q(4), -q(1)^2 - q(2)^2 + q(3)^2 + q(4)^2 ];
60
61 sb(i,:) = R_bi*si;
62 end
63
64 save('sunbody.mat', 'sb', 'x', 't')
65
66 %% Functions
67
68 function dx = qw(t, x)
69 J = J_body;
70
71 dx(1:3,:) = - inv(J)*cross(x(1:3), J*x(1:3));
72 Omega = [0 x(3) -x(2) x(1); -x(3) 0 x(1) x(2); x(2) -x(1) 0 x(3); -x(1) -
73 x(2) -x(3) 0];
74 dx(4:7,:) = 0.5.*Omega*x(4:7);
75 end

```

B.3 Main Kalman filter script

This is a concise version of the main Kalman Filter script which has been use to test the algorithm.

```

1 clear; clc; close all
2
3 % AD & GNC code
4 % Vehicle's equation of motion
5 % Maria Luisa Scarpa - GDP MTV04
6
7 %% Dynamic equation of motion - Body frame
8
9 % J - satellite moment of inertia
10 % w_b - angular velocity of the body fram relative to the inertial frame
11 % w_b_dot - time derivative of the above
12 % T_a - actuation component of torque in body frame
13 % T_p - perturbation component of torque in body frame
14
15 J = J_body;
16
17 %syms J [3 3]
18 syms w_b [3 1]
19 syms T_a [3 1]
20 syms T_p [3 1]
21
22 w_b_dot = inv(J)*(T_a + T_p - cross(w_b, J*w_b));
23
24
25 %% Kinematic equation of motions using quaternions
26

```

```

27 % q – attitude expressed in quaternion form body rate with respect to
    orbit
28 % frame
29
30 syms q [4 1]
31 syms n
32
33
34 Omega = [0 w_b(3) -w_b(2) w_b(1); -w_b(3) 0 w_b(1) w_b(2); w_b(2) -w_b(1)
    0 w_b(3); -w_b(1) -w_b(2) -w_b(3) 0];
35 q_dot = 0.5.*Omega*q;
36
37
38 %% Rotation matrix express in quaternion form
39 R_bi= [q(1)^2-q(2)^2-q(3)^2+q(4)^2, 2*(q(1)*q(2) + q(3)*q(4)), 2*(q(1)*q
    (3) ...
40     - q(2)*q(4)); 2*(q(1)*q(2) - q(3)*q(4)), -q(1)^2 + q(2)^2 - q(3)^2 +
    ...
41     q(4)^2, 2*(q(2)*q(3) + q(2)*q(4)); 2*(q(1)*q(3) + q(2)*q(4)), 2*q(2)*
    q(3) ...
42     - q(1)*q(4), -q(1)^2 - q(2)^2 + q(3)^2 + q(4)^2 ];
43
44
45 %% Extended Kalman filter design and State Space
46
47 % Matrices initialisation
48 % P –estimate error covariance matrix
49 % Q – process noise covariance matrix
50 % R – measurement noise covariance matrix
51 % dt – time step for discretisation (Seconds)
52 % https://apps.dtic.mil/dtic/tr/fulltext/u2/a430928.pdf
53
54 dt = 0.01;
55
56
57 R = diag([0.1^2 0.1^2 0.1^2]);
58
59 %% EXAMPLE TESTING
60 %sunbody.mat has the results from ODE45s which gives the actual
    trajectory
61 %with time as well sun vector in inertial frame.
62 load('sunbody.mat')
63 % real intial conditions
64 q_eq_r = [1 0 0 0]';
65 w_eq_r = [0.5 0.5 0.5]';
66
67 % randomly generated initial conditions
68 q_eq(1) = 0.5*rand + 0.5;
69 q_eq(2) = 0.1*rand;
70 q_eq(3) = 0.1*rand;
71 q_eq(4) = 0.1*rand;
72
73 w_eq(1) = 0.5*rand + 0.5;

```

```

74 w_eq(2) = 0.5*rand + 0.5;
75 w_eq(3) = 0.5*rand + 0.5;
76
77 q_eq = q_eq';
78 w_eq = w_eq';
79
80 P= diag([(q_eq(1)-q_eq_r(1))^2, (q_eq(2)-q_eq_r(2))^2, (q_eq(3)-q_eq_r(3))^2, (q_eq(4)-q_eq_r(4))^2, (w_eq(1)-w_eq_r(1))^2, (w_eq(2)-w_eq_r(2))^2, (w_eq(3)-w_eq_r(3))^2]);
81 Q= diag([0.008^2, 0.008^2, 0.008^2, 0.008^2, 0.008^2, 0.008^2, 0.008^2]);
82
83 si = sunpositionECI('19-May-2021');
84 T_a_eq = [0 0 0]';
85
86 % adding Gaussian Noise to the measurement
87
88 s_noises = sb + 0.1*randn(length(t),3);
89
90 %% Initialisation
91
92 x_new(1:4,1) = q_eq;
93 x_new(5:7,1) = w_eq;
94 cov_err(:, 1) = diag(P);
95
96 %% Filter function
97 for i = 2:length(t)
98     dt = (t(i) - t(i-1))/2;
99
100     [x_new(:,i), P_new] = EKF(q, w_b, q_dot, w_b_dot, q_eq, w_eq, T_a, T_a_eq, dt, P, Q, R, R_bi, s_noises(i,:) ', si);
101
102     cov_err(:,i) = diag(P_new);
103
104     P = P_new;
105
106     q_eq = x_new(1:4, i);
107
108     w_eq = x_new(5:7, i);
109     tstep(i) = dt;
110 end

```

B.4 Non linear controller

This is the script realised to design the non linear PID controller.

```

1 %% Additional controller method using error wrt reference trajectory
2
3 % Design of a non linear controller using Lyapunov stability criterion.
4 % Maria Luisa Scarpa -GDP
5
6
7 % Solving for the quaternions response with time
8 clear; clc

```

```

9
10 % time span for the simulation and initial conditions (errors in
11 % quaternions and initial angular velocity)
12 tspan = [0 3000];
13 x0 = [0.2; 0.4; 0.5; 0.02; 0.01; 0.03; 0; 0; 0; 0; 0; 0];
14
15
16 [t, x] = ode45(@unconstrained, tspan, x0);
17
18
19 x = real(x);
20
21 %% Controller response with time
22
23 % Parameters:
24 % K, P : gains for the feedback PD law
25 % omega_r : desired body rate
26 % omega_r_Dot : desired angular acceleration
27 % J: moment of inertia
28 % T_P : modelled external perturbations
29
30 K = 20*10^2;
31 P = [60*10^3 0 0; 0 60*10^3 0; 0 0 150*10^3];
32 KI = [3 0 0; 0 3 0; 0 0 3];
33
34
35 omega_r = [0 0 0]';
36 omega_r_dot = [0 0 0]';
37 J = [10^6 0 0; 0 10^6 0; 0 0 10^7];
38 T_p = [14, 14, 14]';
39
40
41 for i = 1:length(t)
42
43     u(i,:) = -K*x(i,1:3)' - P*(x(i,4:6)' - omega_r) - KI*x(i,7:9)' + J*
         omega_r_dot - cross(x(i,4:6)', omega_r) ...
         + cross(x(i,4:6)', J*x(i,4:6)') - T_p;
44
45
46
47 % ----- Comment out to add saturation condition -----
48     if abs(u(i,1)) > 2000
49         u(i,1) = 2000*sign(u(i,1));
50
51     elseif abs(u(i,2)) > 2000
52         u(i,2) = 2000*sign(u(i,2));
53
54     elseif abs(u(i,3)) > 2000
55         u(i,3) = 2000*sign(u(i,3));
56
57
58 end
59 % %-----
60 end

```

```

61
62 save('u_input.mat','u','x','t')
63
64 %% Pole placement and linearisation analysed
65
66 %closed loop matrix
67
68 A = [zeros(3) 0.5*eye(3); -K*inv(J) -inv(J)*P];
69
70 % Poles, damping ratio and natural frequency calculations
71
72 for i = 1:3
73
74     poles(i) = 1/(2*J(i,i))*(-P(i,i) + sqrt(P(i,i)^2 - 2*K*J(i,i)));
75
76     wn(i) = 1/(2*J(i,i))*sqrt(2*K*J(i,i)-2*P(i,i)^2);
77
78     zeta(i) = P(i,i)/(sqrt(2*K*J(i,i)-2*P(i,i)^2));
79
80     T(i) = 2*J(i,i)/P(i,i);
81
82     wd(i) = 1/(2*J(i,i))*sqrt(2*K*J(i,i)-P(i,i)^2);
83
84
85 end
86
87
88
89
90 %% Plotting results
91
92
93 % Attitude error
94 figure
95 hold on
96 grid on
97 set(gca,'GridLineStyle','—')
98 plot(t,x(:,1),'b','Linewidth',1.2)
99 plot(t,x(:,2),'r','Linewidth',1.2)
100 plot(t,x(:,3),'k','Linewidth',1.2)
101
102 xlabel('time [secs]')
103 ylabel('Attitude quaternions error with time')
104 legend('q1','q2','q3')
105
106
107 % Angular velocity
108 figure
109 hold on
110 grid on
111 set(gca,'GridLineStyle','—')
112 plot(t,x(:,4),'b','Linewidth',1.2)
113 plot(t,x(:,5),'r','Linewidth',1.2)

```

```

114 plot(t, x(:,6), 'k', 'Linewidth', 1.2)
115 xlabel('time [secs]')
116 ylabel('Angular velocity with time [rad/secs]')
117 legend('\omega_x', '\omega_y', '\omega_z')
118
119 % Control law
120 figure
121 hold on
122 grid on
123 set(gca, 'GridLineStyle', '—')
124 plot(t, u(:,1), 'b', 'Linewidth', 1.2)
125 plot(t, u(:,2), 'r', 'Linewidth', 1.2)
126 plot(t, u(:,3), 'k', 'Linewidth', 1.2)
127 xlabel('time [secs]')
128 ylabel('Control torque input [Nm]')
129 legend('u_x', 'u_y', 'u_z')
130
131
132
133 function dx = unconstrained(t, x)
134
135 % Definition of set of nonlinear differential equations
136 % Specifications:
137 % omega_r : desired body rate
138 % omega_r_Dot : desired angular acceleration
139 % J: moment of inertia
140 % T_P : modelled external perturbations
141 % T_p-2 : unmodelled disutbrances torque
142
143
144 omega_r = [0 0 0]';
145 omega_r_dot = [0 0 0]';
146 J = [10^6 0 0; 0 10^6 0; 0 0 10^7];
147 T_p = [14 14 14]';
148 T_p2 = [5 5 -5]';
149
150 % u = -K*quat_err - P*omega_r
151 % K, P : gains associated
152 % kI : integral gain to remove steady state error
153
154 K = 20*10^2;
155 P = [60*10^3 0 0; 0 60*10^3 0; 0 0 150*10^3];
156 KI = [3 0 0; 0 3 0; 0 0 3];
157 B = 54.73;
158 % ———Useful relationships———:
159
160 q4 = sqrt(1-x(1)^2 - x(2)^2 - x(3)^2);
161 q_tilt = [ 0 -x(3) x(2); x(3) 0 -x(1); -x(2) x(1) 0];
162 u = - K*x(1:3) -P*(x(4:6) - omega_r) -KI*x(7:9) + J*omega_r_dot - cross(x
    (4:6), omega_r) - T_p + cross(x(4:6), J *x(4:6));
163
164
165

```

```

166
167 %—— Comment out to addd saturation conditions
168 if abs(u) > 2000
169     u = 2000*sign(u);
170 end
171
172 %—————
173
174 % set of differential equations
175 dx(1:3, :) = 0.5*(q4*eye(3) + q_tilt)*(x(4:6) - omega_r);
176 dx(4:6, :) = inv(J)*(u + T_p + T_p2 - cross(x(4:6), J*x(4:6)));
177 dx(7:9, :) = x(1:3);
178 dx(10:12, :) = -u - cross(x(4:6), x(10:12));
179
180
181 end

```

B.5 CMGs

Finally, the following code has been created in order to estimate the gimbal angles required for a specific maneuver. The singularity avoidance law is also introduced.

```

1 %% CMG Analysis
2 clear; clc
3
4 syms g_a [4 1]
5 syms h_d [3 1]
6 load('u_input.mat')
7
8 b = 54.73;
9 g_a_n(1,:) = [0 0 0 0];
10
11 A = [-cosd(b)*cosd(g_a(1)), sind(g_a(2)), +cosd(b)*cosd(g_a(3)), -sind(
    g_a(4));...
12     -sind(g_a(1)), -cosd(b)*cosd(g_a(2)), sind(g_a(3)), cosd(b)*cosd(g_a
    (4));...
13     sind(b)*cosd(g_a(1)), sind(b)*cosd(g_a(2)), sind(b)*cosd(g_a(3)),
    sind(b)*cosd(g_a(4))];
14
15 A_inv = pinv(A);
16
17 M(1) = 0;
18
19 for i = 1 : length(t)
20
21     h_dot(i, :) = -u(i, :) - cross(x(i, 4:6), x(i, 10:12));
22
23 end
24
25
26 for i = 2 : length(t)
27
28     dt = t(i) - t(i-1);
29

```



```

30     A_new = double(subs(A, g_a, g_a_n(i-1,:)'));
31
32     M(i) = sqrt(det(A_new*A_new'));
33
34     if M(i) > 1
35
36         rho = 0;
37
38     elseif 0.1/M(i) < 0.2
39
40         rho = 0.1/M(i);
41     else
42         rho = 0.2;
43     end
44
45     H = sqrt(x(i,10)^2 + x(i,11)^2 + x(i,12)^2)/4;
46     g_a_n(i,:) = inv(H)*A_new'*pinv(A_new*A_new' + rho*eye(3))*h_dot(i
        -1,:)'*dt + g_a_n(i-1,:)';
47
48
49
50
51 end
52
53
54 %% Calculation of gimbal rate
55
56
57 figure
58 hold on
59 plot(t, g_a_n(:,1), 'r', 'Linewidth', 1.2);
60 plot(t, g_a_n(:,2), 'b', 'Linewidth', 1.2);
61 plot(t, g_a_n(:,3), 'k', 'Linewidth', 1.2);
62 plot(t, g_a_n(:,4), 'Linewidth', 1.2);
63 xlabel('time [secs]');
64 ylabel('Gimbal angles');
65 legend('\delta_1', '\delta_2', '\delta_3', '\delta_4')

```